

Bankr Skill Reference

Non-custodial Perpetual DEX on Arbitrum · Bankr AI Agent Integration

API BASE	https://og.nexustradinglabs.com
APP	https://trade.nexustradinglabs.com
CHAIN	Arbitrum One — chainId: 42161
BROKER	nexus_trading (Orderly Network broker)
THESIS REGISTRY	0x2F4EdA890f96a7979d6f26bCB210cEDAD68346Bc
REP SCORE	0xAaEE9BF647252Df40ec32eAF6dA29804863483Fe
ORDERLY VAULT	0x816f722424B49Cf1275cc86DA9840Fbd5a6167e9
USDC (Arbitrum)	0xaf88d065e77c8cC2239327C5EDb3A432268e5831
VERIFY CONTRACT	0x6F7a338F2aA472838dEFD3283eB360d4Dff5D203 (mainnet EIP-712 domain)

Nexus is a cypherpunk-grade perp DEX. Trading keys are derived server-side from wallet signatures — no private keys ever leave the user's wallet. Every public thesis is registered on the ThesisRegistry contract on Arbitrum — wins and losses are trustless, verifiable, and immutable.

■ CRITICAL — Trade Flow (NON-NEGOTIABLE)

Every trade goes through this exact 3-step sequence. Skipping any step will fail.

STEP 1

Request sign_message tool

```
request_additional_tools({ names: ['sign_message'] })
commit_to_tools({ keep: ['sign_message'] })
```

STEP 2

Get wallet signature (EVERY trade, no exceptions)

```
sign_message({ message: 'nexus-trading-key-v1' })
→ returns walletSig hex string
Also capture walletAddress.
```

STEP 3

POST /trade

```
POST https://og.nexustradinglabs.com/trade
{ symbol, side, notional, leverage, walletSig, walletAddress }
```

The server derives the ed25519 trading key from walletSig via sha256 → PKCS8 → Ed25519. Fully server-side. Non-custodial by design.

■ Trading Endpoints

POST /trade

symbol	PERP_BTC_USDC, PERP_ETH_USDC, PERP_SOL_USDC, PERP_HYPE_USDC, PERP_ARB_USDC, PERP_XMR_USDC, ...
side	'BUY' (long) or 'SELL' (short)
notional	USD notional size, e.g. 50 = \$50 position
leverage	Integer multiplier, e.g. 5 = 5x
walletSig	Hex signature from <code>sign_message('nexus-trading-key-v1')</code>
walletAddress	User's connected wallet address
stopLoss	Optional — SL trigger price (MARK_PRICE)
takeProfit	Optional — TP trigger price (MARK_PRICE)

POST /set-sl-tp — attach SL/TP AFTER fill (always separate from /trade)

symbol	Same symbol as the open position
stopLoss	Optional — SL trigger price
takeProfit	Optional — TP trigger price
walletSig	Same walletSig from step 2
walletAddress	User wallet

Server fetches current position qty from Orderly, places POSITIONAL_TP_SL algo order (child_orders with CLOSE_POSITION type, close side = opposite of position direction).

Natural language → trade mapping

INPUT	PARAMETERS
'Short HYPE \$20 at 20x'	<code>{ symbol: PERP_HYPE_USDC, side: SELL, notional: 20, leverage: 20 }</code>
'Long BTC \$50 at 5x, SL 60k, TP 75k'	POST /trade first, then POST /set-sl-tp with stopLoss/takeProfit
'Close my SOL short'	POST /trade { symbol: PERP_SOL_USDC, side: BUY, notional: same, leverage: same }

■ Registration Flow (one-time per wallet)

Triggered when /trade returns { error: 'wallet_not_registered' } (HTTP 401).

Step R1 — Ask user for Bankr API key

Tell the user: "Your wallet isn't linked to a Nexus trading account yet. I need your Bankr API key to complete the one-time setup — find it at [bankr.bot/api](#) (enable 'Wallet & Agent API')."

Step R2 — POST /proxy/bankr-register

Body: { walletAddress, bankrApiKey, depositAmount: 5 }
Server: checks Orderly account, EIP-712 signs Registration via Bankr Wallet API,
signs AddOrderlyKey, registers ed25519 key with Orderly, stores { accountId, orderlyKey } in KV.

Step R3 — Retry original trade

Immediately retry POST /trade with same walletSig and walletAddress. No user action needed.

No deposits or on-chain transactions required for registration.

Withdrawal Flow

POST /proxy/bankr-withdraw

walletAddress	User's connected wallet
bankrApiKey	User's Bankr API key (transient only)
amount	USDC amount to withdraw (number)

Server-side flow:

1. Derives ed25519 key from Bankr `personal_sign('nexus-trading-key-v1')`
 2. Fetches withdrawal nonce from Orderly `/v1/withdraw_nonce`
 3. Builds EIP-712 Withdraw message
 4. Signs via Bankr `eth_signTypedData_v4`
 5. POSTs to Orderly `/v1/withdraw_request`
- Returns { ok: true, amount, withdrawNonce } on success

CRITICAL — EIP-712 verifyingContract for Orderly mainnet (Arbitrum, chainId 42161):
0x6F7a338F2aA472838dEFD3283eB360d4Dff5D203 Wrong: 0xCcCCccccCCCCcCcCcCcCCCCcCCCCccccC
(off-chain placeholder → code 29) Wrong: 0x8794E7260517B1766fc7b55cAfd56e6bf08600e (dev env →
code 29) This address must appear in BOTH the EIP-712 signing domain AND the POST body. Source:
mainnetVerifyAddress in @orderly.network/core (networkId === 'mainnet' branch).

Withdraw only `free_collateral` — not total account value. `cost_position` (unsettled fees/funding) is locked margin even with no open position — it clears automatically.

Confirmed withdrawal: 19 USDC · withdraw_id 260510081400063 · wallet 0xd9f7b327... · May 10 2026 ■

Deposit Flow

Simple path — POST /deposit/prepare (recommended for agents)

wallet	User's wallet address
amount	USDC amount
accountId	Fetches automatically from Orderly API — never compute manually

Returns two ready-to-sign transactions: Step 1 = USDC approve to vault, Step 2 = `vault.deposit()`. Sign and submit in order. Funds available within ~1 block (~2s on Arbitrum).

Manual path — raw contract calls

Vault	0x816f722424B49Cf1275cc86DA9840Fbd5a6167e9
USDC	0xaf88d065e77c8cC2239327C5EDb3A432268e5831 (6 decimals)
brokerHash	0x69729be60357fd58653e988388922e200193543b4328eda1b9b9bdaaef2f1a70
tokenHash	0xd6aca1be9729c13d677335161321649cccae6a591554772516700f986f942eaa

DO NOT recompute brokerHash/tokenHash — use exactly as-is.

■ Full API Reference

METHOD	ENDPOINT	DESCRIPTION
POST	/trade	Open/close perp position
POST	/set-sl-tp	Attach SL/TP to open position (after fill)
POST	/proxy/bankr-register	One-time wallet registration via Bankr
POST	/proxy/bankr-withdraw	Withdraw USDC via Bankr EIP-712 signing
POST	/deposit/prepare	Get signed deposit txs (approve + vault.deposit)
GET	/feed	All public theses (status, copyCount, onChainId)
GET	/lab/:walletAddress	All theses for a wallet
PUT	/lab/:walletAddress	Save full thesis array for wallet
GET	/profile/:walletAddress	Trader profile { pfp, displayName }
PUT	/profile/:walletAddress	Update profile
GET	/positions?wallet=&sig;=	Open positions for wallet
GET	/balance?wallet=&sig;=	Account balance and margin
GET	/wallets/onchain	All wallets ever registered on-chain (Arbitrum RPC)
GET	/og/trader/:walletAddress	SVG preview card (og:image)
GET	/notifications/:walletAddress	Fetch notifications
POST	/notifications/:walletAddress	Create notification
PUT	/notifications/:walletAddress	Mark all read
GET	/comments/:thesisId	Fetch comments
POST	/comments/:thesisId	Post comment
GET	/reactions/:thesisId	Fetch emoji reactions
PUT	/reactions/:thesisId/:emoji	Toggle reaction (👍👎👏👀👊)

■ Rep Score

Composite 0–100 on-chain credibility score. Derived from verifiable ThesisRegistry data.

```
repScore = winRate + min(avgRR * 10, 20) - samplePenalty
winRate = (wins / (wins + losses)) * 80
avgRR = mean((tp1 - entry) / (entry - sl)) for closed wins
samplePenalty = 0 if ≥5 closed trades,
else (5 - closed) * 4
Clamped to [0, 100]. Zero closed trades → score = 0.
```

On-chain stats (trustless):

Call `getTraderStats(walletAddress)` on ThesisRegistry → returns (wins, losses, activeTrades). RANKS leaderboard re-sorts using this data. ■ verified badge shown for wallets with on-chain theses.

■ ThesisRegistry Contract

ADDRESS	0x2F4EdA890f96a7979d6f26bCB210cEDAD68346Bc (Arbitrum One)
ARBISCAN	https://arbiscan.io/address/0x2F4EdA890f96a7979d6f26bCB210cEDAD68346Bc
TESTNET	0x6eE20a021F83c7ddD9FA3d15ed6798fCF6A6f468 (Arbitrum Sepolia)
COMPILER	Solidity 0.8.34, via_ir = true

Key functions:

FUNCTION	SIGNATURE	NOTES
registerThesis	(symbol, direction, entry, sl, tp) → thesisId	Prices * 1e8. Called on set-public.
closeThesis	(thesisId, outcome, settlementTxHash)	outcome: HIT_TP STOPPED_OUT CLOSED
getTraderStats	(address) → (wins, losses, activeTrades)	View call — free, no gas
getThesis	(thesisId) → ThesisData	Full thesis struct

Price encoding: `Math.round(price * 1e8)` — e.g. BTC at \$65,432.10 → 6543210000000

Parse ThesisRegistered event from tx receipt to get thesisId: `parseEventLogs(abi, { eventName: 'ThesisRegistered' })`

■ Copy Trading Flow

1. GET /feed → find thesis by trader/symbol/direction
2. Prompt user for: account size (USDC), risk % (e.g. 2%), max loss cap (optional)
3. Compute position size: `accountSize * riskPct / (entry - sl)` [for longs]
4. GET /lab/:userWallet → fetch current theses
5. Append new thesis: all levels from source, copiedFromWallet, copiedThesisId, isPublic: false, attribution in notes
6. PUT /lab/:userWallet → save
7. Optionally POST /notifications/:sourceWallet → follow alert

■ Leaderboard Query

1. GET /feed → all public theses
2. Group by wallet. Compute: winRate (HIT_TP = win, STOPPED_OUT = loss), avgRR, repScore
3. Sort by repScore desc, then total theses desc
4. For ■ verified stats: call getTraderStats(wallet) on ThesisRegistry

■ Never Do

- NEVER use the Orderly CLI (@orderly.network/cli)
- NEVER skip sign_message — walletSig is derived from signature, not stored anywhere
- NEVER ask user for API keys/secrets except Bankr key during registration
- NEVER attempt manual message signing or HMAC auth
- NEVER install npm packages or run shell commands to place a trade
- NEVER store the Bankr API key — transient only
- NEVER pass amount as String in EIP-712 uint256 fields — must be a number
- NEVER assume Cloudflare 1101 = Orderly 1101 — check Content-Type (HTML = CF crash, JSON = Orderly error)
- NEVER withdraw total account value — withdraw only free_collateral

■ Example Agent Prompts

"Show me the top 5 traders on Nexus by Rep Score"

→ GET /feed → group by wallet → compute repScore → sort desc → return top 5

"Short HYPE \$20 at 20x"

→ sign_message → POST /trade { symbol: PERP_HYPE_USDC, side: SELL, notional: 20, leverage: 20 }

"Long BTC \$50 at 5x, SL 60k, TP 75k"

→ sign_message → POST /trade → POST /set-sl-tp { stopLoss: 60000, takeProfit: 75000 }

"Withdraw 19 USDC"

→ Ask for Bankr API key → POST /proxy/bankr-withdraw { amount: 19 }

"What's my Rep Score?"

→ GET /feed → filter by walletAddress → compute repScore with formula → return score + breakdown

"Copy top Nexus trader's active BTC thesis at 2% risk"

→ Build leaderboard → get top wallet → GET /feed → find PERP_BTC_USDC thesis → compute size → PUT /lab

"Publish my ETH short: entry 3200, SL 3350, TP 2800"

→ registerThesis('PERP_ETH_USDC', 'SHORT', 320000000000, 335000000000, 280000000000) → PUT /lab

■ Infra Notes

WORKER	nexus-lab-api — deployed at og.nexustradinglabs.com (Cloudflare custom domain). KV: LAB_STORE (id: 12c4fcbc04cc4211902d49b8abc2a751).
DEPLOY	git push → GitHub Actions → Cloudflare Pages (frontend) + Worker (nexus-lab-api). Worker deploy added to .github/workflows/deploy.yml (npx wrangler@3 deploy).

GIT LOCKS	If .git/index.lock blocks commits in sandbox: clone to /tmp, push via PAT. Or: del .git\index.lock in Windows PowerShell.
PAT SCOPE	GitHub PAT needs 'workflow' scope to push deploy.yml changes via API. Without it, edit deploy.yml directly in GitHub UI.
CF 1101	Cloudflare error 1101 = Worker JS exception → HTML response. Orderly error 1101 = invalid EIP-712 sig → JSON response. Check Content-Type.

Nexus Trading Labs — Non-custodial Perpetual DEX on Arbitrum. Your keys, your funds. Code is law.